SQL Interview Questions and Answers

Intermediate to advanced level (about 2 years of experience)
Simple answers you can say in your own words

How to use this: Read each question, answer out loud first, then compare with the answer here. In SQL interviews you will often be asked to write the query live, so try to type each query yourself. The 'Say in interview' box is a short version you can speak in 20 to 30 seconds. SQL syntax is a little different in each database. These examples use PostgreSQL style, so tell the interviewer which database you have used. These are practice questions based on common topics, not leaked questions from any company.

Sample tables used in the examples

```
employees(emp_id, name, salary, dept_id, manager_id, hire_date)
departments(dept_id, dept_name)
customers(customer_id, name, city)
orders(order_id, customer_id, order_date, amount, category)
logins(user_id, login_date)
```

Part 1: Core Concepts You Must Know Well

Q1. What is the order of execution of a SQL query? What is the difference between WHERE and HAVING?

Answer:

The order we write a query is not the order the database runs it. The database runs it like this: first FROM and JOIN, then WHERE, then GROUP BY, then HAVING, then SELECT, then DISTINCT, then ORDER BY, and last LIMIT.

- **WHERE** filters the rows before grouping. It cannot use aggregate functions like AVG.
- **HAVING** filters the groups after grouping. It can use aggregate functions.
- Because SELECT runs after WHERE, we cannot use a column alias from SELECT inside WHERE.

```
SELECT dept_id, AVG(salary) AS avg_sal
FROM employees
WHERE hire_date >= '2023-01-01'      -- filter rows first
GROUP BY dept_id
HAVING AVG(salary) > 50000;          -- then filter groups
```

Say in interview: The database runs FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY, LIMIT in this order. WHERE filters rows before grouping and HAVING filters groups after grouping.

Q2. Explain the types of joins. How do you find customers who have no orders?

Answer:

A join combines rows from two tables on a matching column.

- **INNER JOIN:** only the matching rows from both tables.
- **LEFT JOIN:** all rows from the left table, and matching rows from the right. No match gives NULL.
- **RIGHT JOIN:** the same, but for the right table.
- **FULL OUTER JOIN:** all rows from both tables.

- **CROSS JOIN:** every row with every row (all combinations).
- **SELF JOIN:** a table joined with itself, like employee and manager.

Example: To find customers with no orders, use a LEFT JOIN and keep the rows where the right side is NULL. This is called an anti-join. NOT EXISTS also works.

```
SELECT c.customer_id, c.name
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;

-- same with NOT EXISTS
SELECT c.customer_id, c.name
FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

Say in interview: INNER keeps only matches, LEFT keeps everything from the left table, FULL keeps everything from both. To find missing rows I use LEFT JOIN with IS NULL, or NOT EXISTS.

Q3. How does SQL handle NULL? What is the NOT IN trap?

Answer:

NULL means unknown, not zero and not empty. Any comparison with NULL gives unknown, so = NULL never works. We must use IS NULL or IS NOT NULL. Aggregate functions like SUM and AVG ignore NULL values, but COUNT(*) counts the rows and COUNT(column) skips NULLs. COALESCE gives a default value when the value is NULL.

Example: The NOT IN trap: if the list inside NOT IN has even one NULL, the whole query returns no rows. This is a very common bug. Use NOT EXISTS instead.

```
-- returns NO rows if orders.customer_id has any NULL
SELECT * FROM customers
WHERE customer_id NOT IN (SELECT customer_id FROM orders);

-- safe version
SELECT * FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);

SELECT COALESCE(city, 'Unknown') FROM customers;
```

Say in interview: NULL means unknown, so I compare with IS NULL. Aggregates ignore NULL. NOT IN breaks if the list contains a NULL, so I prefer NOT EXISTS.

Q4. What is the difference between UNION and UNION ALL? What are INTERSECT and EXCEPT?

Answer:

UNION joins the results of two queries and removes duplicates, so it needs extra work and is slower. UNION ALL joins them and keeps all rows, so it is faster. If we know there are no duplicates, or we want them, use UNION ALL. INTERSECT gives only the rows present in both results. EXCEPT gives the rows in the first result that are not in the second. The columns must match in number and type.

Say in interview: UNION removes duplicates and UNION ALL keeps them and is faster. INTERSECT gives common rows and EXCEPT gives the rows only in the first query.

Q5. What is the difference between DELETE, TRUNCATE, and DROP?

Answer:

DELETE removes selected rows using WHERE. It is logged row by row, so it is slower, and it can fire triggers.

- **TRUNCATE** removes all the rows quickly and usually resets the auto-increment counter. It has no WHERE.
- **DROP** removes the whole table, including its structure.
- Whether TRUNCATE can be rolled back depends on the database, so say that you would check it.

Say in interview: DELETE removes chosen rows, TRUNCATE empties the whole table fast, and DROP removes the table itself.

Q6. What are keys and constraints? What is normalization?

Answer:

Constraints keep the data correct: PRIMARY KEY (unique and not null, identifies a row), FOREIGN KEY (points to a primary key in another table), UNIQUE, NOT NULL, and CHECK.

- **Normalization** means organizing tables to avoid repeated data.
- **1NF:** each cell has one value, no lists inside a cell.
- **2NF:** every column depends on the whole primary key.
- **3NF:** columns depend only on the key, not on other non-key columns.
- **Denormalization:** adding repeated data on purpose to make reads faster. It is common in analytics.

Say in interview: Keys and constraints protect the data. Normalization removes repeated data by splitting tables, and I denormalize only when I need faster reads.

Part 2: Window Functions (asked in almost every interview)

Q7. What are window functions? How are they different from GROUP BY?

Answer:

GROUP BY collapses many rows into one row per group. A window function does a calculation across a group of related rows, but it keeps every row and adds the result as a new column. The group is defined by OVER with PARTITION BY (the groups) and ORDER BY (the order inside the group).

```
SELECT name, dept_id, salary,
       AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg
FROM employees;      -- every employee row, plus their dept average
```

Say in interview: GROUP BY reduces the rows. A window function keeps all the rows and adds a calculated column, like a rank or a running total, for each row's group.

Q8. What is the difference between ROW_NUMBER, RANK, and DENSE_RANK?

Answer:

They all number the rows in order. The difference is how they treat ties.

- **ROW_NUMBER:** always unique numbers. Ties get different numbers.
- **RANK:** ties get the same number, and then a gap. Example: 1, 1, 3.

- **DENSE_RANK:** ties get the same number, and no gap. Example: 1, 1, 2.

Example: Salaries 100, 100, 90 give ROW_NUMBER 1, 2, 3. RANK gives 1, 1, 3. DENSE_RANK gives 1, 1, 2.

Say in interview: ROW_NUMBER is always unique, RANK leaves gaps after ties, and DENSE_RANK has no gaps.

Q9. How do you find the Nth highest salary?

Answer:

The safest way is DENSE_RANK, because it handles ties correctly. Another way is ORDER BY with LIMIT and OFFSET, together with DISTINCT.

```
-- 3rd highest salary
SELECT DISTINCT salary
FROM (SELECT salary,
             DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
      FROM employees) t
WHERE rnk = 3;

-- another way
SELECT DISTINCT salary FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 2;
```

Say in interview: I use DENSE_RANK ordered by salary descending and pick rank N. It handles ties correctly.

Q10. How do you find the top 3 highest paid employees in each department?

Answer:

This is the classic top N per group question. We use a window function with PARTITION BY the department, then filter the rank in an outer query. We cannot filter on a window function directly in WHERE, because WHERE runs before the window function.

```
SELECT * FROM (
    SELECT e.*,
           DENSE_RANK() OVER (PARTITION BY dept_id
                              ORDER BY salary DESC) AS rnk
    FROM employees e) t
WHERE rnk <= 3;
```

Say in interview: I rank the employees inside each department with DENSE_RANK and PARTITION BY, then filter the rank in an outer query.

Q11. What are LAG and LEAD? How do you calculate month-over-month growth?

Answer:

LAG gives the value from a previous row, and LEAD gives the value from a next row, in the window order. They are used to compare a row with the one before it, like this month against last month.

```
WITH monthly AS (
    SELECT DATE_TRUNC('month', order_date) AS month,
           SUM(amount) AS revenue
    FROM orders
    GROUP BY 1)
SELECT month, revenue,
       LAG(revenue) OVER (ORDER BY month) AS prev_revenue,
       ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
             / LAG(revenue) OVER (ORDER BY month), 2) AS growth_pct
FROM monthly;
```

Say in interview: LAG reads the previous row's value. I group the revenue by month, use LAG to get last month, and then calculate the percent change.

Q12. How do you calculate a running total and a moving average?

Answer:

We use SUM or AVG as a window function with ORDER BY. For a moving average we also give a frame, which tells how many rows to include, for example the current row and the 6 rows before it.

```
SELECT order_date, amount,
       SUM(amount) OVER (ORDER BY order_date) AS running_total,
       AVG(amount) OVER (ORDER BY order_date
           ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS moving_avg_7
FROM daily_sales;
```

Say in interview: A running total is SUM with OVER and ORDER BY. A moving average adds a ROWS BETWEEN frame, like 6 PRECEDING AND CURRENT ROW.

Q13. How do you find users who logged in for 3 or more days in a row? (gaps and islands)

Answer:

This is a famous hard question. The trick: give each login date a row number per user. If the dates are consecutive, then date minus row number gives the same value for the whole streak. So rows with the same value belong to the same streak. Then we group by that value and count. First we remove duplicate days with DISTINCT.

```
WITH d AS (
    SELECT DISTINCT user_id, login_date FROM logins),
g AS (
    SELECT user_id, login_date,
           login_date - CAST(ROW_NUMBER() OVER (
               PARTITION BY user_id ORDER BY login_date) AS INT) AS grp
    FROM d)
SELECT user_id, MIN(login_date) AS start_date,
       MAX(login_date) AS end_date, COUNT(*) AS streak_days
FROM g
GROUP BY user_id, grp
HAVING COUNT(*) >= 3;
```

Say in interview: I number each user's login dates, subtract the row number from the date, and the result is the same for a consecutive streak. Then I group by it and count.

Part 3: Subqueries, CTEs and Tricky Queries

Q14. What is the difference between a subquery, a CTE, and a correlated subquery?

Answer:

A **subquery** is a query inside another query. A **CTE** (WITH clause) gives a name to a temporary result at the top of the query, so the query is easier to read and the result can be reused. A **correlated subquery** uses a column from the outer query, so it runs again for every outer row. It can be slow on big tables, and often a join or a window function is faster.

```
-- correlated: employees earning more than their department average
SELECT e.name, e.salary
FROM employees e
WHERE e.salary > (SELECT AVG(salary) FROM employees
                  WHERE dept_id = e.dept_id);

-- faster window version
SELECT name, salary FROM (
    SELECT name, salary,
           AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg
    FROM employees) t
WHERE salary > dept_avg;
```

Say in interview: A subquery is a query inside a query. A CTE names a temporary result and makes the query readable. A correlated subquery depends on the outer row and runs once for each of them, so it can be slow.

Q15. What is the difference between EXISTS and IN? Which is better?

Answer:

IN checks if a value is in a list of values from the subquery. EXISTS checks if the subquery returns at least one row, and it stops at the first match. EXISTS is usually better when the subquery result is big, and it is safe with NULLs. IN is fine for small lists. Modern databases often optimize both in the same way, so check the execution plan.

Say in interview: EXISTS checks if any matching row exists and stops early. It is safe with NULL and good for big subqueries. IN is fine for short lists.

Q16. What is a recursive CTE? How do you show an employee hierarchy?

Answer:

A recursive CTE is a CTE that calls itself. It has two parts joined with UNION ALL: the starting query (the anchor, like the CEO who has no manager), and the recursive part that finds the next level (the employees who report to the previous level). It stops when no new rows are found. It is used for org charts, categories, and folder trees.

```
WITH RECURSIVE org AS (
    SELECT emp_id, name, manager_id, 1 AS level
    FROM employees
    WHERE manager_id IS NULL                -- anchor: the top
    UNION ALL
    SELECT e.emp_id, e.name, e.manager_id, o.level + 1
    FROM employees e
    JOIN org o ON e.manager_id = o.emp_id)  -- next level
SELECT * FROM org ORDER BY level;
```

Say in interview: A recursive CTE has an anchor query for the top level, and a recursive part that repeatedly joins to find the next level, until there are no more rows.

Q17. How do you find the employees who earn more than their manager?

Answer:

We use a self join. We join the employees table with itself: one copy as the employee, and one copy as the manager, matched on manager_id.

```
SELECT e.name AS employee, e.salary, m.name AS manager, m.salary
FROM employees e
JOIN employees m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

Say in interview: I do a self join of the employee table, matching the employee's manager_id to the manager's emp_id, and compare the two salaries.

Q18. How do you find and delete duplicate rows, keeping one copy?

Answer:

To find them, group by the columns that define a duplicate and use HAVING COUNT greater than 1. To delete them and keep one, number the rows inside each duplicate group with ROW_NUMBER, and delete the rows with number greater than 1.

```
-- find
SELECT email, COUNT(*) FROM users
GROUP BY email HAVING COUNT(*) > 1;

-- delete, keep the lowest id
DELETE FROM users
WHERE id IN (
    SELECT id FROM (
        SELECT id, ROW_NUMBER() OVER (PARTITION BY email
                                      ORDER BY id) AS rn
        FROM users) t
    WHERE rn > 1);
```

Say in interview: I find duplicates with GROUP BY and HAVING COUNT greater than 1. To delete, I use ROW_NUMBER partitioned by the duplicate columns and delete the rows numbered above 1.

Q19. How do you find customers who ordered in both January and February?

Answer:

We filter the two months, group by customer, and count the distinct months. If the count is 2, the customer ordered in both months.

```
SELECT customer_id
FROM orders
WHERE order_date >= '2026-01-01' AND order_date < '2026-03-01'
GROUP BY customer_id
HAVING COUNT(DISTINCT DATE_TRUNC('month', order_date)) = 2;
```

Say in interview: I filter the two months, group by customer, and keep the customers who have 2 distinct months.

Q20. How do you turn rows into columns (pivot) in SQL?

Answer:

Many databases do not have an easy PIVOT, so we use conditional aggregation: a CASE WHEN inside SUM or COUNT, for each column we want. The same idea is used to count different statuses in one query.

```
SELECT customer_id,
       SUM(CASE WHEN EXTRACT(MONTH FROM order_date) = 1
                THEN amount ELSE 0 END) AS jan,
       SUM(CASE WHEN EXTRACT(MONTH FROM order_date) = 2
                THEN amount ELSE 0 END) AS feb
FROM orders
GROUP BY customer_id;
```

Say in interview: I use SUM with CASE WHEN for each output column. This is called conditional aggregation.

Q21. How do you calculate each category's percentage of total revenue?

Answer:

We sum the revenue by category, and divide by the total revenue. The total can be taken with a window function over the whole result, using an empty OVER (). This avoids a separate subquery.

```
SELECT category,
       SUM(amount) AS revenue,
       ROUND(100.0 * SUM(amount) / SUM(SUM(amount)) OVER (), 2) AS pct
FROM orders
GROUP BY category
ORDER BY revenue DESC;
```

Say in interview: I group by category and divide each sum by the overall sum, which I get with SUM of SUM OVER an empty window.

Part 4: Performance and Optimization

Q22. What is an index? What are the types, and when should you not use one?

Answer:

An index is a separate structure, like the index at the back of a book, that helps the database find rows fast without reading the whole table. The common type is a B-tree index.

- **Clustered:** decides how the rows are physically stored. A table can have only one.
- **Non-clustered:** a separate structure that points to the rows. A table can have many.
- **Composite index:** on many columns. The order matters. The index on (customer_id, order_date) helps queries on customer_id alone, or on both, but not on order_date alone (the leftmost prefix rule).
- **Cost:** indexes use space and make INSERT, UPDATE and DELETE slower. Do not index small tables, or columns with very few different values, or columns that are rarely used in filters.

```
CREATE INDEX idx_orders_cust_date
ON orders (customer_id, order_date);
```

Say in interview: An index lets the database find rows without scanning the whole table. It speeds up reads but slows down writes and uses space, so I index the columns used often in WHERE, JOIN and ORDER BY.

Q23. What does 'sargable' mean? Which query is better, and why?

Answer:

A condition is sargable when the database can use an index for it. If we apply a function on the indexed column, the index cannot be used, and the database scans the full table. So keep the column clean on the left side, and put the calculation on the other side.

```
-- slow: function on the column, index not used
SELECT * FROM orders WHERE EXTRACT(YEAR FROM order_date) = 2026;

-- fast: range on the raw column, index can be used
SELECT * FROM orders
WHERE order_date >= '2026-01-01' AND order_date < '2027-01-01';
```

Say in interview: A sargable condition can use an index. I avoid putting functions on the indexed column, and I use a range condition instead.

Q24. A query is running slow. How do you debug it?

Answer:

I follow these steps:

- Run **EXPLAIN** (or EXPLAIN ANALYZE) to see the plan. Look for full table scans on big tables, and a big difference between the estimated and the actual rows.
- Check if the right **indexes** exist for the filters and joins.
- Check for functions on indexed columns, SELECT *, and too many rows or columns returned.
- Check the join order and the join size. Filter early, before the join, where possible.
- Replace correlated subqueries with joins or window functions.
- Update the table statistics if they are old.
- Add LIMIT, or use partitioning for very big tables. Check if the app is calling the query too many times.

Say in interview: I look at the EXPLAIN plan first, find the full scans, check the indexes and the filters, remove the functions on columns, and test again.

Q25. What join algorithms does a database use?

Answer:

The database chooses one for each join by itself:

- **Nested loop:** for each row on one side, look up the other side. Good for small tables, or when there is an index on the join column.
- **Hash join:** builds a hash table of the smaller table, and probes it with the bigger one. Good for large tables with an equality join.
- **Merge join:** both sides are sorted, and then merged. Good when the data is already sorted, or on big ranges.

Say in interview: The database picks between nested loop, hash join, and merge join. Nested loop suits small or indexed lookups, hash join suits big equality joins, and merge join suits sorted data.

Q26. What is partitioning and what is sharding?

Answer:

Both split a big table into smaller parts. **Partitioning** splits one table into parts inside the same database, for example by month. Queries that filter by month only read the needed partition. **Sharding** splits the data across different servers, for example by customer id, so that the load is shared. Sharding gives more scale, but it makes joins and transactions across shards harder.

Say in interview: Partitioning splits a big table inside one database, usually by date. Sharding splits the data across many servers to scale.

Part 5: Transactions and Database Design

Q27. What is a transaction? What is ACID?

Answer:

A transaction is a group of operations that must work as one unit: all of them succeed, or none. Example: moving money. We take it from account A and add it to account B. If the second step fails, the first must be undone.

- **Atomicity:** all or nothing.
- **Consistency:** the data always follows the rules and constraints.
- **Isolation:** transactions running together do not disturb each other.
- **Durability:** after commit, the data is saved even if the system crashes.

```
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;      -- or ROLLBACK if something failed
```

Say in interview: A transaction is a group of steps that succeed or fail together. ACID means atomicity, consistency, isolation, and durability.

Q28. What are isolation levels? What problems do they prevent?

Answer:

Isolation levels decide how much one running transaction can see of the other transactions. The problems are:

- **Dirty read:** reading data that another transaction has not committed yet.
- **Non-repeatable read:** reading the same row twice and getting different values, because someone changed it in between.
- **Phantom read:** running the same query twice and getting different rows, because someone added or removed rows.
- The levels, from weakest to strongest: Read Uncommitted, Read Committed, Repeatable Read, Serializable. Stronger means safer but slower. PostgreSQL uses Read Committed by default, and MySQL InnoDB uses Repeatable Read.

Say in interview: Isolation levels control what a transaction can see of other transactions. Higher levels prevent dirty, non-repeatable, and phantom reads, but are slower.

Q29. What is a deadlock? How do you avoid it?

Answer:

A deadlock happens when two transactions each hold a lock that the other one needs, so both wait forever. The database detects it and cancels one of them with an error.

- Always access the tables and rows in the same order.
- Keep transactions short.
- Use proper indexes, so fewer rows are locked.
- Retry the transaction in the application when a deadlock error comes.

Say in interview: A deadlock is two transactions waiting for each other's locks. I avoid it by using the same lock order, short transactions, and a retry in the code.

Q30. What is the difference between a view and a materialized view? And procedure, function, and trigger?

Answer:

A **view** is a saved query that works like a virtual table. It runs again each time we use it. A **materialized view** stores the result physically, so reading is fast, but the data can be old until we refresh it.

- **Stored procedure:** a saved block of SQL logic that we call, can do inserts and updates, and can control transactions.
- **Function:** returns a value and can be used inside a query.
- **Trigger:** runs automatically when an event like INSERT, UPDATE, or DELETE happens. Use it carefully, as it can hide logic and slow down writes.

Say in interview: A view is a saved query, and a materialized view stores the result and needs a refresh. A procedure is called to do work, a function returns a value, and a trigger runs by itself on a change.

Q31. What is the difference between OLTP and OLAP? What are star schema and SCD?

Answer:

OLTP systems handle day-to-day transactions, like orders and payments. Many small reads and writes, normalized tables. **OLAP** systems are for analysis and reports. Fewer but heavy queries that read a lot of data, and denormalized tables.

- **Star schema:** one big fact table (events and numbers, like sales) in the center, connected to smaller dimension tables (customer, product, date).
- **SCD (Slowly Changing Dimension)** is how we handle changes in dimension data. Type 1 overwrites the old value. Type 2 adds a new row and keeps the history with start and end dates. Type 3 keeps the old value in an extra column.

Say in interview: OLTP is for daily transactions and OLAP is for analysis. A star schema has a central fact table with dimension tables around it, and SCD Type 2 keeps the full history of changes.

Q32. SQL vs NoSQL. When do you use which?

Answer:

SQL databases have tables with a fixed structure, strong consistency, and joins. They are best for structured data and transactions, like orders and banking. NoSQL databases are more flexible in structure and scale out easily. They are good for huge data, changing structure, or special data shapes: document (MongoDB), key-value (Redis), column (Cassandra), and graph (Neo4j).

Say in interview: I use SQL for structured data that needs transactions and joins. I use NoSQL for flexible or huge data that needs easy scaling.

Part 6: SQL in AI and LLM Work

Q33. How can an LLM be used with SQL? (text-to-SQL) What are the risks?

Answer:

In text-to-SQL, the user asks a question in plain English, and the LLM writes the SQL query. We give the LLM the table and column names (the schema) and a few example queries in the prompt, then run the query and show the result.

- The LLM can write a wrong query, or a very heavy one.
- It can write a dangerous query like DELETE or DROP. So use a **read-only** database user, allow only SELECT, and add LIMIT and a timeout.
- Validate the query before running, and log everything.
- Give only the needed tables to the LLM, to avoid exposing private data, and add row-level access control.

Say in interview: The LLM writes the SQL from the question and the table schema. I run it with a read-only user, only SELECT, with a limit and timeout, and I validate it first.

Q34. What is pgvector? How does a SQL database help in RAG?

Answer:

pgvector is an extension for PostgreSQL that lets us store embeddings in a column, and search by similarity (nearest vectors) with SQL. The advantage is that the vectors live next to our normal data. We can filter by normal columns like department or date in the same query, and use the same backups and permissions. For very large scale or special features, a dedicated vector database like Pinecone may be better.

Say in interview: pgvector adds vector storage and similarity search to PostgreSQL, so I can keep embeddings next to my normal data and filter with normal SQL.

If you do not know an answer, say honestly: 'I have not worked on this directly, but my understanding is...' and then explain what you know. It is better than guessing. For query questions, think aloud and explain your steps. Interviewers care about your approach.